



Лабораторная работа № 5
"Символьный экран и матричная клавиатура"

Группа: Z3301

Студент: Вестфаль Мишель, Карасев Георгий, Ушмакова Дарья

Преподаватель: Денисов Константин Михайлович

1 Цель

Реализовать систему ввода пароля с помощью матричной клавиатуры и вывода сообщений на символьный экран

2 Задачи

1. Реализовать функции взаимодействия с дисплеем (инициализация, управление командами)
2. Реализовать функцию взаимодействия с клавиатурой
3. Реализовать протокол взаимодействия микропроцессора и ПК с помощью интерфейса UART

3 Блок схема кода

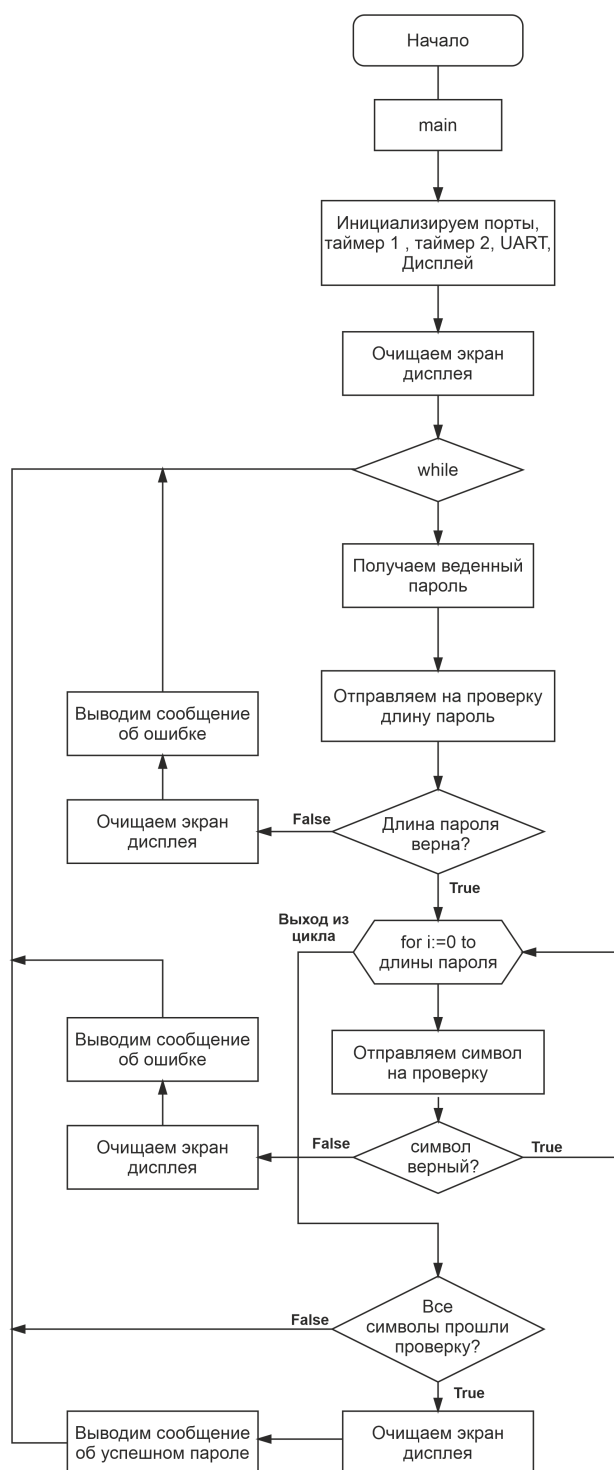


Рис. 1: Блок схема основной части кода

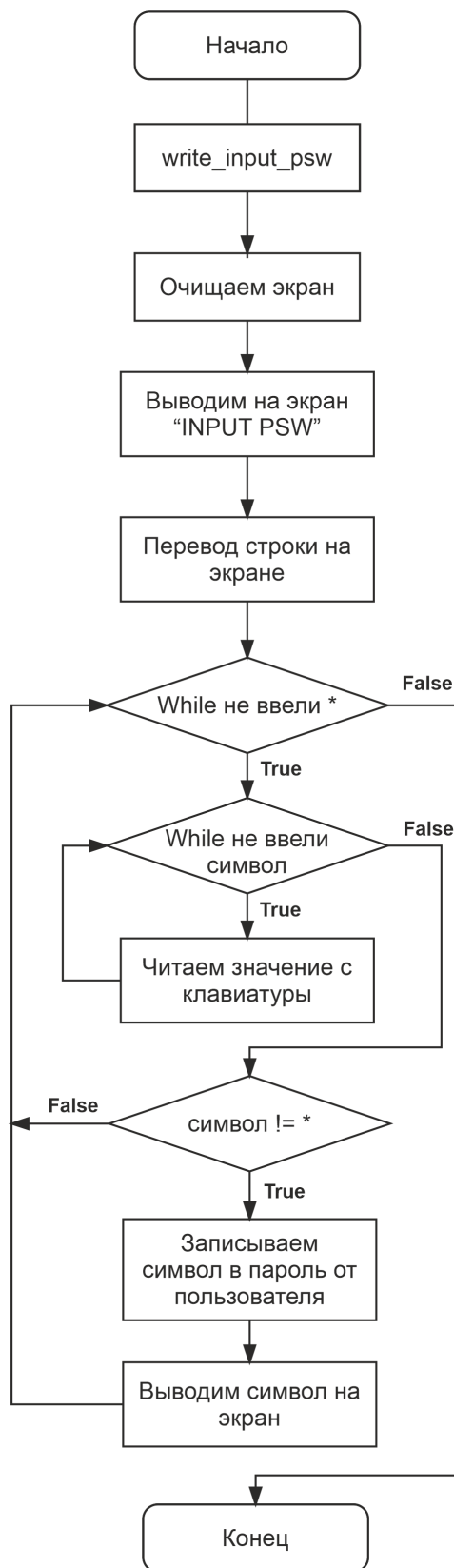


Рис. 2: Блок схема получения пароля от пользователя

4 Пояснение к реализации

4.1 Общая структура программы

Программа реализует систему ввода и проверки пароля на базе микроконтроллера C8051F360 с использованием символьного дисплея, матричной клавиатуры 4×4 и интерфейса UART.

Пользователь вводит пароль с клавиатуры, введенные символы отображаются на дисплее, после чего пароль передается по UARTу для проверки. В зависимости от результата проверки на дисплей выводится сообщение о правильности или неправильности пароля.

Основными задачами программы являются:

- инициализация микроконтроллера и периферийных устройств;
- организация временных задержек;
- работа с символьным дисплеем;
- опрос матричной клавиатуры;
- ввод и хранение пароля;
- обмен данными через UART;
- отображение результата проверки пароля.

4.2 Инициализация глобальных переменных

`delay_counter` используется для формирования временных задержек.

`len_line` задаёт максимальную длину строки дисплея и равна 16 символам.

`input_type` определяет текущий режим ввода символов с клавиатуры.

`input_len_psw` хранит длину введенного пароля.

`max_len_psw` задаёт максимально допустимую длину пароля.

Массив `input_psw` предназначен для хранения введенного пользователем пароля.

Также в программе определены символьные массивы:

- `incorrect_password` — сообщение о неправильном пароле;
- `correct_password` — сообщение о правильном пароле;
- `input_password` — приглашение к вводу пароля.

4.3 Настройка линий ввода-вывода

Для обращения к отдельным выводам микроконтроллера используются битовые переменные.

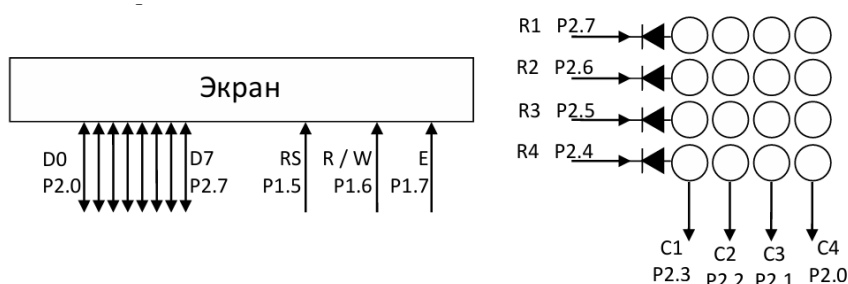


Рис. 3: Символьный экран и матричная клавиатура

Для управления символьным дисплеем используются сигналы:

- $RS = P1^5$ — единичный уровень - запись(чтение) кода символа на текущее знакоместо, а нулевой - запись команды управления в дисплей или чтение текущего состояния экрана;
- $R_W = P1^6$ — единичный уровень означает чтение из дисплея, а нулевой - запись в дисплей;
- $E = P1^7$ — формирование нулевого импульса при реализации чтения либо записи информации в или из дисплея.

4.4 Инициализация микроконтроллера

В функции `main()` выполняется начальная настройка микроконтроллера и периферийных устройств. Вызываются функции:

- `PORT_Init();`
- `Timer_Init();`
- `UART_Init();`
- `Timer2_Init();`
- `Timer2_ISR();`
- `Display_Init();`

`PORT_Init()` выполняет настройку портов ввода-вывода и кроссбар-интерфейса микроконтроллера. Регистр `XBR0` включает интерфейс `UART`, а `XBR1` активирует кроссбар.

4.5 Настройка таймеров

Для работы `UART` используется `Timer1`. Таймер `Timer2` настраивается на генерацию прерываний с частотой 1000 Гц, то есть каждые 1 мс.

4.6 Обработчик прерывания `Timer2`

Прерывание `Timer2` используется для реализации программной задержки.

В обработчике прерывания:

```
void Timer2_ISR (void) interrupt 5
```

сбрасывается флаг переполнения таймера:

```
TF2H = 0;
```

Если значение `delay_counter` не равно нулю, оно уменьшается на единицу:

```
delay_counter--;
```

Поскольку прерывание возникает каждые 1 мс, уменьшение счётчика соответствует отсчёту времени в миллисекундах.

4.7 Функция задержки delay()

Для корректной работы дисплея и клавиатуры необходимо выдерживать временные интервалы между операциями.

Функция delay() реализует программную задержку на основе Timer2:

```
void delay(unsigned int time)
```

При вызове функции переменной delay_counter присваивается значение времени ожидания в миллисекундах:

```
delay_counter = time;
```

После этого программа переходит в цикл ожидания:

```
while(delay_counter != 0){}
```

Значение счётчика уменьшается в обработчике прерывания Timer2_ISR(), поэтому после достижения нуля выполнение программы продолжается.

4.8 Инициализация дисплея

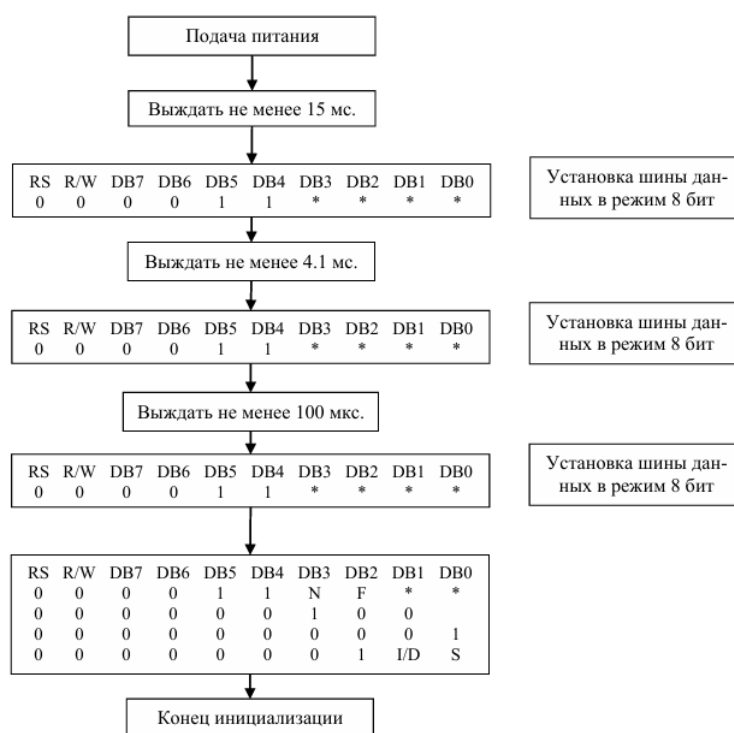


Рис. 4: Блок схема алгоритма инициализации дисплея

4.9 Функции взаимодействия с дисплеем

Для работы с дисплеем реализованы функции:

- send_command_to_display();
- send_char_to_display();
- send_word_to_display_first_line();

- `send_word_to_display_second_line()`.

Функция `send_command_to_display()` используется для передачи управляющих команд дисплею.

При передаче команды:

- RS устанавливается в 0;
- R_W устанавливается в 0;
- на порт P2 передаётся код команды;
- формируется импульс сигнала E.

Функция `send_char_to_display()` работает аналогично, однако RS устанавливается в 1, что соответствует передаче данных.

Функции `send_word_to_display_first_line()` и `send_word_to_display_second_line()` выполняют последовательный вывод строки символов на дисплей.

Для перехода к первой строке используется адрес DDRAM 0x00, для второй строки — адрес 0x40.

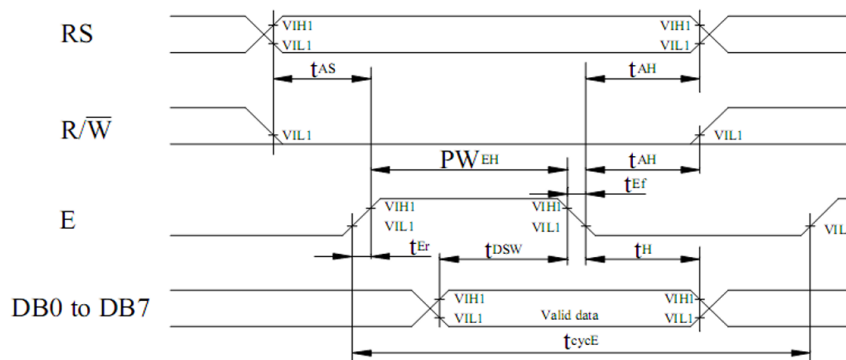


Рис. 5: Цикл записи информации в дисплей

4.10 Функции взаимодействия через UART

Для организации обмена данными с компьютером используются функции:

- `UART_Init();`
- `uart_send_char();`
- `uart_read_char();`

Функция `UART_Init()` выполняет настройку UART:

`SCON0 = 0x10;`

Функция `uart_send_char()` записывает передаваемый символ в регистр SBUF0 и ожидает установки флага завершения передачи TI0.

Функция `uart_read_char()` ожидает установки флага приёма RI0, после чего считывает принятый символ из регистра SBUF0.

Через UART выполняется передача длины пароля и последующая посимвольная передача введённого пароля для проверки.

4.11 Считывание матричной клавиатуры

Для работы с клавиатурой реализованы функции:

- `get_value_from_key_board()`;
- `read_keyboard()`.

Функция `get_value_from_key_board()` выполняет построчное сканирование клавиатуры. Сначала все линии переводятся в логическую единицу:

```
P2 = 0xff;
```

После небольшой задержки строки клавиатуры поочерёдно переводятся в логический ноль.

После активации строки проверяется состояние столбцов C1–C4. Если на одном из столбцов обнаруживается нулевой уровень, определяется нажатая клавиша.

В зависимости от значения `input_type` клавиши могут соответствовать различным наборам символов.

Кнопки последнего столбца переключают режимы ввода:

- режим 0 (цифры);
- режим 1 (первые буквы);
- режим 2 (вторые буквы);
- режим 3 (третьи буквы).

Функция `read_keyboard()` дополнительно реализует ожидание отпускания клавиши:

```
while((P2 & 0x0F) != 0x0F){}
```

Это предотвращает многократное считывание одного нажатия.

```
1 // Includes
2
3
4 #include <c8051F360.h> // SFR declarations
5 #include <domofon.h>
6
7
8 // 16-bit SFR Definitions for 'F36x
9
10 sfr16 TMR2RL = 0xCA; // Timer2 reload value
11 sfr16 TMR2 = 0xCC; // Timer2 counter
12
13
14 #define SYSCLK 3062500
15
16 // Global value
17
18
19 unsigned int delay_counter = 0;
20 unsigned char len_line = 16; // in symbols
21 unsigned char input_type = 0;
22
23 unsigned char input_len_psw = 0;
```



```

24 unsigned char max_len_psw = 16;
25
26
27 sbit R1 = P2^7;
28 sbit R2 = P2^6;
29 sbit R3 = P2^5;
30 sbit R4 = P2^4;
31
32 sbit C1 = P2^3;
33 sbit C2 = P2^2;
34 sbit C3 = P2^1;
35 sbit C4 = P2^0;
36
37 sbit RS = P1^5;
38 sbit R_W = P1^6;
39 sbit E = P1^7;
40
41 unsigned char input_psw[16];
42
43 unsigned char incorrect_password[] = {'P', 'S', 'W', ' ', 'I', 'S', ' ', ' ',
    'I', 'N', 'C', 'O', 'R', 'E', 'C', 'T'};
44 unsigned char correct_password[] = {'P', 'S', 'W', ' ', 'I', 'S', ' ', ' ',
    'C', 'O', 'R', 'E', 'C', 'T'};
45 unsigned char input_password[] = {'I', 'N', 'P', 'U', 'T', ' ', ' ', 'P', 'S',
    'W'};
46
47 char command;
48
49
50 // Function PROTOTYPES
51
52
53 void PORT_Init (void);
54 void Timer2_Init (int counts);
55 void Timer2_ISR (void);
56 void Display_Init(void);
57 void UART_Init(void);
58 void Timer_Init();
59
60 void delay(unsigned int time);
61
62 void send_command_to_display(unsigned char command);
63 void send_char_to_display(unsigned char symbol);
64 void send_word_to_display_first_line(unsigned char* word, char len);
65 void send_word_to_display_second_line(unsigned char* word, char len);
66
67 unsigned char read_keyboard(void);
68 unsigned char get_value_from_key_board(void);
69
70 void uart_send_char(char c);
71 char uart_read_char(void);
72
73 void write_input_psw(void);

```

```

74
75 // MAIN Routine
76
77 void main (void)
78 {
79     unsigned char res = 0;
80     unsigned char i = 0;
81     PCA0MD &= ~0x40;                // WDTE = 0 (clear watchdog timer
82                                     // enable)
83
84     PORT_Init();
85     Timer_Init();
86     UART_Init();
87     Timer2_Init (SYSCLK / 12 / 1000); // Init Timer2 to generate
88                                     // at a 1000 Hz rate.
89
90 // UART_Init();
91
92     EA = 1;                        // Enable global interrupts
93
94     Display_Init();
95
96     SFRPAGE = LEGACY_PAGE;        // Page to sit in for now
97
98     send_command_to_display(0x01);
99
100    while (1) {
101        // input psw
102        write_input_psw();
103
104        // check len psw
105        uart_send_char(input_len_psw);
106        res = uart_read_char();
107
108        if (res == 0xaa) {
109            send_command_to_display(0x01);
110            send_word_to_display_first_line(incorrect_password,
111            sizeof(incorrect_password)/sizeof(incorrect_password[0]));
112            delay(5000);
113        } else {
114            if (res == 0x55) {
115                for (i = 0; i < input_len_psw; i++) {
116                    uart_send_char(input_psw[i]);
117                    res = uart_read_char();
118                    if (res == 0xaa) {
119                        send_command_to_display(0x01);
120                        send_word_to_display_first_line(incorrect_password,
121                        sizeof(incorrect_password)/sizeof(incorrect_password[0]));
122                        delay(3000);
123                        break;
124                    }
125                }
126            }
127        }
128    }
129 }

```

```

124
125     if(i == input_len_psw) { // get to end {
126         send_command_to_display(0x01);
127         send_word_to_display_first_line(correct_password,
128 sizeof(correct_password)/sizeof(incorrect_password[0]));
129         delay(3000);
130     }
131 }
132 } // Spin forever
133 }
134
135
136 // PORT_Init
137
138 void PORT_Init (void)
139 {
140     unsigned char SFRPAGE_save = SFRPAGE; // Save the current SFRPAGE
141
142     SFRPAGE = CONFIG_PAGE; // Switch to the necessary SFRPAGE
143
144     XBR0      = 0x01;
145     XBR1      = 0x40; // Enable crossbar and weak pull-ups
146
147     SFRPAGE = SFRPAGE_save; // Restore the SFRPAGE
148 }
149
150 // Timer2_Init
151
152 void Timer2_Init (int counts)
153 {
154     unsigned char SFRPAGE_save = SFRPAGE; // Save the current SFRPAGE
155
156     SFRPAGE = CONFIG_PAGE; // Switch to the necessary SFRPAGE
157
158     TMR2CN = 0x00; // Stop Timer2; Clear TF2;
159 // Use SYSCLK/12 as timebase
160     TMR2RL = -counts; // Init reload values
161     TMR2 = 0xFFFF; // Set to reload immediately
162
163     ET2 = 1; // Enable Timer2 interrupts
164
165     TR2 = 1; // Start Timer2
166
167     SFRPAGE = SFRPAGE_save; // Restore the SFRPAGE
168 }
169
170 void Timer_Init()
171 {
172     TCON      = 0x40;
173     TMOD      = 0x20;
174     CKCON     = 0x08;
175     TH1       = 0x60;

```

```

176 }
177
178 // UART_Init
179
180 void UART_Init(void)
181 {
182     SCON0      = 0x10;
183 }
184
185 // Display_Init
186
187 void Display_Init(void) {
188
189     delay(15);
190     send_command_to_display(0x30); // 0011 0000
191     delay(5);
192     send_command_to_display(0x30); // 0011 0000
193     delay(1);
194     send_command_to_display(0x30); // 0011 0000
195     delay(1);
196     send_command_to_display(0x38); // 0011 1000
197     delay(1);
198     send_command_to_display(0x08); // 0000 1000
199     delay(1);
200     send_command_to_display(0x01); // 0000 0001
201     delay(1);
202     send_command_to_display(0x06); // 0000 0110
203     delay(1);
204     send_command_to_display(0x0e); // 0000 1110
205     delay(1);
206     send_command_to_display(0x18); // 0001 1000
207 }
208
209 // Timer2_ISR
210
211 void Timer2_ISR (void) interrupt 5
212 {
213     TF2H = 0;
214
215     if (delay_counter != 0) {
216         delay_counter--;
217     }
218     // Clear Timer2 interrupt flag
219 }
220
221 // delay
222
223 void delay(unsigned int time){
224     delay_counter = time;
225     while(delay_counter != 0) {
226
227     }
228 }

```

```

229 }
230
231
232 // send_command_to_display()
233
234 void send_command_to_display(unsigned char command){
235     E = 0;
236     RS = 0;
237     R_W = 0;
238     P2 = command;
239     delay(1);
240     E = 1;
241     delay(1);
242     E = 0;
243     delay(1);
244 }
245
246
247 // send_char_to_display()
248
249 void send_char_to_display(unsigned char symbol){
250     E = 0;
251     RS = 1;
252     R_W = 0;
253     P2 = symbol;
254     delay(1);
255     E = 1;
256     delay(1);
257     E = 0;
258     delay(1);
259 }
260
261
262 // send_command_to_display()
263
264 void send_word_to_display_first_line(unsigned char* word, char len){
265     int i = 0;
266
267     if (len > len_line) len = len_line;
268
269     send_command_to_display(0x80);
270
271     for (i = 0; i < len; i++){
272         send_char_to_display(word[i]);
273     }
274 }
275
276 void send_word_to_display_second_line(unsigned char* word, char len){
277     int i = 0;
278
279     if (len > len_line) len = len_line;
280
281     send_command_to_display(0xc0);

```

```

282
283     for (i = 0; i < len; i++){
284         send_char_to_display(word[i]);
285     }
286 }
287
288
289 // uart_read_char
290
291 char uart_read_char(void){
292     while (RIO == 0){};
293     command = SBUF0;
294     RIO = 0;
295     return command;
296 }
297
298
299 // uart_send_char
300
301 void uart_send_char(char c){
302     SBUF0 = c;
303     while (TIO == 0){};
304     TIO = 0;
305 }
306
307
308 // read_keyboard
309
310 unsigned char get_value_from_key_board(void) {
311     P2 = 0xff;
312     delay(10); // time to charge
313
314     R1 = 0;
315     if (C1 == 0) {
316         return first_tab[input_type];
317     }
318     if (C2 == 0) {
319         return second_tab[input_type];
320     }
321     if (C3 == 0) {
322         return third_tab[input_type];
323     }
324     if (C4 == 0) {
325         input_type = 0;
326         return ' ';
327     }
328     R1 = 1;
329
330     R2 = 0;
331     if (C1 == 0) {
332         return fourth_tab[input_type];
333     }
334     if (C2 == 0) {

```

```

335     return fifth_tab[input_type];
336 }
337 if (C3 == 0) {
338     return sixth_tab[input_type];
339 }
340 if (C4 == 0) {
341     input_type = 1;
342     return ' ';
343 }
344 R2 = 1;
345
346 R3 = 0;
347 if (C1 == 0) {
348     return seventh_tab[input_type];
349 }
350 if (C2 == 0) {
351     return eighth_tab[input_type];
352 }
353 if (C3 == 0) {
354     return ninth_tab[input_type];
355 }
356 if (C4 == 0) {
357     input_type = 2;
358     return ' ';
359 }
360 R3 = 1;
361
362 R4 = 0;
363 if (C1 == 0) {
364     return '*';
365 }
366 if (C2 == 0) {
367     return zeroth_tab[input_type];
368 }
369 if (C3 == 0) {
370     return '#';
371 }
372 if (C4 == 0) {
373     input_type = 3;
374     return ' ';
375 }
376 R4 = 1;
377 return ' ';
378 }
379
380 unsigned char read_keyboard(void) {
381     unsigned char res = get_value_from_key_board();
382     while((P2 & 0x0F) != 0x0F) {}; // Wait for the buttons to be underpressed
383     return res;
384 }
385
386 void write_input_psw(void){
387     unsigned char res = ' ';

```

```

388 unsigned char i = 0;
389
390 send_command_to_display(0x01);
391
392 send_word_to_display_first_line(input_password, sizeof(input_password)
/ sizeof(input_password[0]));
393
394 send_command_to_display(0xc0);
395
396 while(res != '*') {
397     res = ' ';
398     while( res == ' '){
399         res = read_keyboard();
400     }
401     if (res != '*') {
402         send_char_to_display(res);
403         input_psw[i] = res;
404         i++;
405     }
406 }
407 input_len_psw = i;
408 }
409
410
411 // End Of File

```

Листинг 1: main.c